

Ch. 5 - InputOutput

Categorie di **dispositivi I/O**:

- *Dispositivi a blocchi*: Archiviazione di informazioni in blocchi di dimensioni fisse, come dischi magnetici o SSD. Ogni blocco può essere letto/scritto indipendentemente.
- *Dispositivi a caratteri*: Flusso di caratteri senza struttura a blocchi, non indirizzabili e senza opzioni di ricerca, come stampanti o interfacce di rete.
- *Altre categorie*: Modelli utili per astrarre alcuni SW di I/O nel SO.

Il FS gestisce i dispositivi a blocchi astratti mentre i SW di livello inferiore gestiscono le specificità dei dispositivi.

I dispositivi I/O sono composti da una **parte meccanica** (il dispositivo stesso) ed una parte **elettronica** (*controller*, che viene spesso integrato nella scheda madre o come scheda aggiuntiva su slot PCIe, e che può spesso gestire più dispositivi identici). Il compito del controller è quello di *convertire il flusso seriale in blocchi di byte, correggere errori, trasferire in memoria principale*.

Senza il controller il programmatore dovrebbe gestire dettagli complessi. Il controller è inizializzato con parametri essenziali, che gli permettono di gestire autonomamente questi dettagli.

Porte:

- *Porta parallela*: Comunemente dotata di una connessione a 25 o 36 pin, trasmette dati multi-bit simultaneamente su più pin. Deprecata
- *Porta USB*: Universal Serial Bus, interfaccia standardizzata per connessione e comunicazione tra dispositivi e computer, utilizzata per trasferire dati e fornire alimentazione elettrica.
 - Supporta connessioni ad alte velocità di trasferimento e viene utilizzato anche per periferiche come tastiere, mouse e dispositivi di archiviazione. Grande facilità d'utilizzo con connessione plug-and-play, retrocompatibile con versioni precedenti, assicura ampio supporto per vari dispositivi

Comunicazione CPU-Dispositivo

Registri dei Controller:

- Ogni controller di dispositivo ha registri per comunicare con la CPU
- La *scrittura nei registri* avviene tramite comandi inviati ai dispositivi, mentre la *lettura nei registri* avviene tramite verifica dello stato del dispositivo e della prontezza per i nuovi comandi.

Buffer di dati:

- Molti dispositivi includono un buffer di dati per scrittura/lettura del SO. Il SO è cruciale per gestire efficientemente trasferimento dati e controllo dei dispositivi, incidendo sulla progettazione e sulle prestazioni del sistema.

Port-Mapped I/O

Assegnazione delle porte I/O:

- Ogni registro di controllo ha un numero di porta di I/O associato
- Formano lo spazio delle porte di I/O, accessibile solo dal SO per motivi di protezione

Istruzioni:

- *Lettura*: `IN REG, PORT` - La CPU legge dal registro di controllo `PORT` e salva il risultato in `REG`
- *Scrittura*: `OUT PORT, REG` - La CPU scrive il contenuto di `REG` in un registro di controllo

Separazione: Spazi di indirizzi della memoria e dell'I/O sono distinti e non correlati

Memory-Mapped I/O

Approccio:

- Assegna ad ogni registro di controllo un indirizzo di memoria univoco, quindi i registri di controllo sono mappati nello spazio di memoria.

Vantaggi:

- *Elimina la necessità di istruzioni speciali di I/O* come `IN` o `OUT`.
- I *registri* di controllo possono essere trattati *come variabili in C*, consentendo la scrittura del driver completamente in C
- *Protezione semplificata*: I processi utente non accedono direttamente ai registri di controllo
- *Controllo selettivo dei dispositivi*: Attraverso la gestione delle pagine di memoria, si può dare controllo selettivo su dispositivi specifici,
 - Consente *l'esecuzione di driver di dispositivi in spazi di indirizzo separati*, aumentando la sicurezza e riducendo le dimensioni del kernel

Attenzione con la cache:

- *Rischio di caching dei registri di controllo*: Un registro finisce in cache, la CPU controlla solo la cache e non controlla se è stato modificato dal dispositivo
- Caso limite: Se un ciclo while è in attesa che il contenuto del registro cambi: **Ciclo infinito**
- Necessità di disabilitare selettivamente la cache per alcuni indirizzi
 - Bisogna identificare con precisione quali pagine sono dedicate ai dispositivi HW
 - Gli accessi a queste aree non saranno ottimizzati dalla cache, quindi possono essere più lenti

Gestione indirizzi e architetture del bus:

- *Necessità* per tutti i moduli di memoria e dispositivi I/O di *esaminare ogni riferimento alla memoria*
- *Problemi con bus della memoria separati in architetture*

SOLUZIONI:

- *Tentativi sequenziali (Memory-First)*: Richiesta indirizzata prima alla memoria principale, se fallisce viene inoltrata ad altri bus. È semplice da implementare ma comporta maggiore latenza per accessi I/O
- *Bus Snooping* ("spia"): Un dispositivo sul bus monitora gli indirizzi e reindirizza quelli destinati ai dispositivi I/O. Porta ad accessi I/O più rapidi ma c'è una maggiore complessità HW.

Prestazioni della Memoria

- Il bus della memoria è ottimizzato per la velocità, ma deve gestire sia gli accessi alla memoria principale che ai dispositivi.
- Tentativi sequenziali preservano le prestazioni della memoria

Il Memory-Mapped I/O richiede un bilanciamento tra *prestazioni* e *complessità*.

Ibrido

Combina **Port-Mapped I/O** e **Memory-Mapped I/O**:

- **PMIO**: Utilizza un indirizzamento separato per i dispositivi di I/O con istruzioni dedicate (**IN** e **OUT**)
- **MMIO**: I registri dei dispositivi sono mappati nello spazio di memoria, accessibili con normali istruzioni di memoria (**Load** e **Store**).

Funzionamento:

1. Configurazione tramite PMIO:

- Configurazione iniziale di dispositivi HW
- Usa lo spazio di indirizzi delle porte e istruzioni **IN/OUT**

2. Accesso ai dati tramite MMIO:

- Operazioni ad alta velocità come trasferimenti di dati
- Dispositivi mappati nello spazio di memoria principale

Vantaggi:

1. **Flessibilità**: PMIO per configurazioni semplici e MMIO per operazioni rapide
2. **Ottimizzazione**: MMIO più veloce per trasferimenti dati continui
3. **Compatibilità Legacy**: PMIO consente di supportare dischi vecchi
4. **Separazione logica**: Configurazione ed utilizzo gestiti in modi distinti.

Svantaggi:

1. **Aumento della complessità**: Due modalità = logiche HW e SW complesse
2. **Overhead iniziale**: PMIO può rallentare la configurazione rispetto ad un approccio puramente MMIO
3. **Limitazioni architetturali**: Alcune CPU moderne non supportano PMIO

DMA

Direct Memory Access, permette alla CPU di scambiare dati con i controller dei dispositivi bypassando il trasferimento manuale byte per byte, riducendo lo spreco di tempo della CPU e migliorando l'efficienza del trasferimento dati.

Configurazione HW: Presenza di un controller DMA in molti sistemi, che può gestire trasferimenti a più dispositivi. Spesso situato sulla scheda madre.

Registri e funzionamento del controller: Contiene registri per indirizzi di memoria, conteggi di byte e controlli.

- **Senza DMA** il controller del disco legge i dati e li memorizza nel suo buffer. Dopo aver controllato gli errori, provoca un interrupt ed il SO copia i dati in memoria
- **Con DMA :**

1. La CPU imposta il controller DMA e invia un comando al controller del disco
2. Il controller DMA richiede la lettura al controller del disco
3. Scrittura in memoria da parte del controller del disco
4. Conferma del controller del disco al controller DMA

Ripetizione dei passi 2-4 fino al completamento del trasferimento. DMA invia un interrupt alla CPU al termine del trasferimento.

Variabilità: *Range* da semplici (un trasferimento alla volta) a complessi (trasferimenti multipli simultanei)

Controller DMA complessi:

- Multipli set di registri per canali diversi
- Ogni canale programmabile per trasferimenti specifici
- Capacità di gestire contemporaneamente diversi controller di dispositivi

Gestione dei trasferimenti multipli:

- Selezione del dispositivo successivo post-trasferimento tramite algoritmi prioritari o Round-Robin
- Linee di conferma separate per ogni canale DMA sul bus

Modalità DMA ed interazioni con il bus:

- **Cycle Stealing:** DMA trasferisce una parola per volta rubando cicli alla CPU. Rallenta leggermente la CPU ma permette la condivisione del bus
- **Burst Mode:** DMA ottiene il controllo completo del bus, eseguendo trasferimenti multipli. Efficiente ma può bloccare la CPU per periodi prolungati
- **Fly-By Mode:** DMA trasferisce dati direttamente alla memoria principale senza intermediari, riducendo l'uso del bus ma richiedendo cicli extra per

ogni trasferimento

DMA usa tipicamente **indirizzi fisici**, richiedendo conversione dal SO

Interrupt

Un dispositivo I/O invia un segnale di interrupt alla CPU tramite la linea del bus assegnata. Se non ci sono altri interrupt in corso, il controller gestisce immediatamente l'interrupt, altrimenti il dispositivo è momentaneamente ignorato

Gestione degli interrupt

Il controller assegna un numero alle linee degli indirizzi per specificare il dispositivo che richiede attenzione ed invia un segnale di interruzione alla CPU.

- La CPU interrompe il suo attuale compito
- Il *vettore degli interrupt* punta all'inizio della procedura di servizio degli interrupt corrispondente
- La procedura di servizio *conferma l'interrupt* scrivendo su una porta del controller degli interrupt (per evitare race condition)
- Il Program Counter deve essere salvato per riavviare i processi interrotti
- Le informazioni prelevate dai dispositivi I/O sono *salvate nei registri interni o sullo stack*

Le moderne CPU utilizzano architetture pipeline e superscalari, complicando la gestione degli interrupt.

- **Interrupt precisi**: Il sistema può determinare con esattezza quali istruzioni sono state completate al momento dell'interrupt e quali no
- **Interrupt imprecisi**: Diverse istruzioni vicino al Program Counter si trovano in vari stati di completamento al momento dell'interrupt, rendendo incerto lo stato esatto del programma

Gli interrupt imprecisi rendono il SO più complesso e lento, ed il salvataggio di molte informazioni rallenta il processo di interrupt e ripristino.

Principi del SW di I/O

Obiettivi:

- **Indipendenza dal dispositivo:** Permessi di accesso a diversi dispositivi senza specificare il tipo di dispositivo in anticipo
- **Denominazione Uniforme:** I nomi di file o dispositivi dovrebbero essere stringhe o numeri indipendenti dal dispositivo
- **Gestione degli errori:** Da gestire *il più vicino possibile all'HW*.
- **Trasferimenti sincroni/asincroni:** La maggior parte dell'I/O fisico è asincrono, ma molti programmi lo trattano come sincrono (bloccante).
- **Buffering:** L'utilizzo può influenzare le prestazioni, ma deve essere garantito
- **Dispositivi Condivisibili vs Dedicati:** Dischi o SSD sono spesso condivisi da più utenti, mentre altri come stampanti o scanner sono tipicamente dedicati.

I/O Programmato

La CPU gestisce direttamente tutto il processo di trasferimento dati.

- Il SO copia il buffer in uno spazio del kernel, inviando i caratteri alla stampante uno alla volta, aspettando che questa sia pronta per ogni carattere.

Polling o Busy Waiting: Il SO entra in un ciclo di polling, controllando il registro di stato della stampante ed inviando un carattere alla volta. L'I/O Programmato è *efficace quando il tempo di elaborazione* di un carattere è *breve*, quindi adatto a sistemi embedded dove la CPU non ha altre attività significative.

I/O Guidato dagli Interrupt

Utilizza gli interrupt per segnalare alla CPU quando un dispositivo è pronto per l'elaborazione successiva, riducendo il polling e consentendo alla CPU di eseguire altre operazioni.

I/O Con DMA

Il DMA riduce il numero di interrupt, passando da uno per ogni carattere ad uno per buffer. Libera la CPU per eseguire altre attività durante il trasferimento I/O.

Struttura del SW di I/O

Organizzazione a 4 livelli, in cui ogni livello ha funzioni ed interfacce specifiche

Gestore degli interrupt

- **Blocco dei driver**: Durante I/O, i driver vengono bloccati fino al completamento dell'I/O e all'arrivo dell'interrupt
- **Gestione complessa**: Diversi passaggi, inclusi salvataggio dei registri, impostazione di contesti e conferme al controller degli interrupt
- **Impatto sulla Memoria Virtuale**: Passaggi aggiuntivi per gestire MMU, TLB e cache, aumentando la complessità e i cicli macchina necessari
- **Elaborazione non banale**: Numerosi cicli di CPU e varia notevolmente a seconda del sistema e dell'architettura

1. Salvataggio dei registri
2. Impostazione del contesto
3. Impostazione dello Stack
4. Conferma al controller degli interrupt
5. Copia dei registri nella tavola dei processi
6. Esecuzione dell'ISR
7. Scelta del processo successivo
8. Impostazione del contesto per il nuovo processo
9. Caricamento dei nuovi registri del processo
10. Avvio del nuovo processo

Driver di dispositivi

Gestiscono i dispositivi I/O attraverso registri di dispositivi specifici

- *Diversi per ciascun tipo* di dispositivo, al più gestiscono un tipo o una classe di dispositivi correlati
- *Ogni dispositivo necessita di un codice specifico*, noto come driver di dispositivo, solitamente fornito dal produttore
Di solito fanno parte del kernel del SO per poter accedere ai registri del controller del dispositivo.
- **Interfaccia con il SO**: Permette l'installazione di codice scritto da terze parti (driver)
- **Classificazione dei driver**: In categorie come dispositivi:
 - *A blocchi*
 - *A caratteri*
- **Caricamento dei driver**: Dinamicamente

SW del SO indipendente dal dispositivo

Funge da intermediario tra i driver specifici dei dispositivi e le applicazioni utente. Mira a **semplificare l'interazione con i dispositivi** HW offrendo un'interfaccia uniforme e gestendo operazioni comuni.

Funzioni chiave:

1. **Interfaccia uniforme dei Driver dei dispositivi**
2. **Buffering**
3. **Segnalazione degli errori**
4. **Allocazione e rilascio dei dispositivi dedicati**
5. **Dimensione dei blocchi indipendente dal dispositivo**

Uniformità nell'interfaccia

Uniformità:

- Evita la necessità di modificare il SO ogni volta che viene introdotto un nuovo dispositivo
- Importante per mantenere la consistenza e l'efficienza del sistema
- Risolve il problema di *Interfacce diverse* vs *Interfacce standard*

SW per I/O a livello utente

Include librerie di I/O che semplificano le chiamate di sistema per operazioni di I/O come lettura e scrittura, migliorando l'interfaccia tra le applicazioni utente e il SO. È progettato per gestire in modo efficiente l'interazione tra le applicazioni utente e l'hardware del sistema, fornendo un'interfaccia standardizzata e gestendo le operazioni di I/O in modo da massimizzare le prestazioni complessive del sistema informatico.